RV UNIVERSITY, BENGALURU-59
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

A Mini Project Report On

MERIDIAN DATA

An AI-Powered Natural Language to SQL Database Explorer and DBMS Teaching Tool

Submitted in partial fulfillment for the award of degree of

B.Tech. (Honors)

in

School of Computer Science and Engineering

Submitted By

Ayon Aryan	1RVU23CSE093
Hardik Goel	1RVU23CSE180
Aniket Kumar	1RVU23CSE055
Aditya Kumar	1RVU23CSE029

Under the Guidance of

Prof. Ashwini Mathur
Assistant Professor
School of CSE
RV University, Bengaluru-560059

2025-2026

RV UNIVERSITY, BENGALURU-59
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

Certified that the mini project work titled **"Meridian Data – An AI-Powered Natural Language to SQL Database Explorer"** is carried out by **Ayon Aryan (1RVU23CSE093), Hardik Goel (1RVU23CSE180), Aniket Kumar (1RVU23CSE055)** and **Aditya Kumar (1RVU23CSE029)**, who are bonafide students of RV University, Bengaluru, in partial fulfillment of **Bachelor of Technology (Hons.) in the School of Computer Science and Engineering** of RV University, Bengaluru, during the year 2025-2026. It is certified that all corrections/suggestions indicated for the internal assessment have been incorporated in the mini project report deposited in the departmental library. The mini project report has been approved as it satisfies the academic requirements in respect of mini project work prescribed by the institution for the said degree.

Signature of Guide Prof. Ashwini Mathur	Signature of Program Director Dr. K C Narendra	Signature of Dean Dr. Shobha G

External Viva:

Sl. No.	Name of Examiners	Signature with Date
1		
2		

RV UNIVERSITY, BENGALURU-59
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING
DECLARATION

We, Ayon Aryan, Hardik Goel, Aniket Kumar and **Aditya Kumar**, students of the sixth semester B.Tech (Hons.), SoCSE, RV University, Bengaluru, hereby declare that the mini project titled *'Meridian Data – An AI-Powered Natural Language to SQL Database Explorer'* has been carried out by us and submitted in partial fulfillment of the Bachelor of Technology (Hons.) in the School of Computer Science and Engineering during the year 2025-2026.

Further, we declare that the content of the report has not been submitted previously by anybody or to any other university.

We also declare that any intellectual property rights generated out of this project carried out at RV University will be the property of RV University, Bengaluru, and we will be one of the authors of the same.

Place: Bengaluru

Date:

Name	USN	Signature
Ayon Aryan	1RVU23CSE093	
Hardik Goel	1RVU23CSE180	
Aniket Kumar	1RVU23CSE055	
Aditya Kumar	1RVU23CSE029	

ACKNOWLEDGEMENT

It is a great pleasure for us to acknowledge the assistance and support of many individuals who have been responsible for the successful completion of this project work.

First, we take this opportunity to express our sincere gratitude to the School of Computer Science and Engineering, RV University, for providing us with a great opportunity to pursue our bachelor's degree in this institution.

A special thanks to our Program Director, **Dr. K C Narendra**, and Dean **Dr. Shobha G**, for their continuous support and for providing the necessary facilities with guidance to carry out the mini project work.

We would like to thank our guide **Prof. Ashwini Mathur**, Assistant Professor, School of Computer Science and Engineering, RV University, for sparing her valuable time to extend help in every step of our project work, which paved the way for the smooth progress and fruitful culmination of the project.

We are also grateful to our family and friends who provided us with every requirement throughout the course. We would like to thank one and all who directly or indirectly helped us in the project work.

Ayon Aryan, Hardik Goel,
Aniket Kumar and Aditya Kumar

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

ABBREVIATIONS USED

Abbreviation	Expansion
SQL	Structured Query Language
NL2SQL / NLIDB	Natural Language to SQL / Natural Language Interface to Databases
LLM	Large Language Model
RBAC	Role-Based Access Control
DBMS	Database Management System
API	Application Programming Interface
DDL / DML	Data Definition Language / Data Manipulation Language
PK / FK	Primary Key / Foreign Key
ER	Entity-Relationship
DFD	Data Flow Diagram
UI / SPA	User Interface / Single-Page Application
CSV / PPTX	Comma-Separated Values / PowerPoint Open XML
JSON	JavaScript Object Notation
LPU	Language Processing Unit
IID	Independent and Identically Distributed

ABSTRACT

Relational databases store the majority of the world's structured information, yet querying them requires fluency in SQL — a barrier for students, analysts, and domain experts alike. Meridian Data is a full-stack platform that lets a user explore any database in plain English: it converts a natural-language request into validated, dialect-specific SQL using Large Language Models, executes it safely, and returns the results together with AI-generated insights, charts, and exportable presentations. To produce accurate queries, the system sends a schema-aware prompt — carrying columns, primary keys, foreign keys, indexes, and live sample rows — to an instruction-tuned model (Groq-served Llama 3.3 70B in the cloud, or a local Ollama Mistral model), with automatic cloud-to-local failover, while frequent introspection commands bypass the model entirely and are answered deterministically from a uniform database adapter layer that abstracts eight engines spanning SQLite, PostgreSQL, MySQL, MSSQL, Oracle, MongoDB, Cassandra, and Redis. Every generated statement then passes a two-stage guardrail of query classification and a destructive-action safety filter, followed by role-based authorization, and any write or schema operation is routed to a human-in-the-loop review page with automatic snapshotting before execution.

Evaluated on the bundled sample databases, the system generated correct SQL for natural-language analytics in a measured mean of 0.85 seconds using the Groq backend (median 0.80 seconds), while deterministic introspection commands responded in under twenty milliseconds; destructive operations such as DROP and TRUNCATE were consistently blocked, and write operations were correctly intercepted for human confirmation. Beyond returning rows, the platform produced executive summaries, automatically selected appropriate chart types, generated multi-widget dashboards, and exported formatted PowerPoint reports. Meridian Data thus demonstrates that academically validated components — cross-domain text-to-SQL generation, post-generation validation, role-based access control, and human oversight — can be integrated into a single, safe, and privacy-flexible tool that makes databases approachable without sacrificing control, serving equally as a productivity aid for analysts and as a DBMS teaching instrument for students.

Chapter 1: Introduction

Databases are the backbone of nearly every digital system, yet the Structured Query Language (SQL) that unlocks them remains an obstacle for a large class of users — first-year computer-science students learning database concepts, business analysts who understand their data but not joins, and domain experts who simply want answers. This chapter introduces Meridian Data, motivates the problem it solves, states the project objectives, and outlines the methodology and the organization of the report.

1.1 State of the Art

The state of the art in making databases accessible has shifted decisively from rule-based natural-language interfaces toward neural and, most recently, Large Language Model (LLM) based text-to-SQL generation. Early systems relied on hand-crafted grammars and were brittle and database-specific. The deep-learning era introduced large benchmarks such as WikiSQL and the cross-domain Spider dataset, and architectures such as RAT-SQL and PICARD that tackled schema linking and constrained decoding. The current frontier is in-context learning: general-purpose models such as Llama 3.3 and GPT-4, prompted with the database schema, generate competitive SQL without task-specific training, with techniques like DIN-SQL and DAIL-SQL refining prompt design.

Commercially, this capability now appears inside cloud data warehouses and business-intelligence suites. However, most such offerings are tied to a single proprietary engine, are closed-source, send data to the cloud unconditionally, and rarely expose the safety, authorization, and human-oversight layers that production database access demands. Meridian Data positions itself within this landscape as an open, multi-engine, guardrailed explorer that can run entirely on a local model for privacy-sensitive use.

1.2 Motivation

The motivation for this project arose from a recurring classroom and workplace observation: people are blocked not by a lack of questions about their data, but by the syntax required to ask them. A student who understands that “top customers by revenue” involves a JOIN and an aggregation may still spend an hour debugging dialect-specific syntax. Existing natural-language tools either hallucinate incorrect SQL, silently execute potentially destructive statements, or lock the user into one database vendor.

We were therefore motivated to build a tool that is (i) trustworthy — it never executes a destructive or unauthorized statement without explicit human confirmation; (ii) universal — it speaks to relational and NoSQL engines through one interface; (iii) educational — it shows the generated SQL and explains it, turning every query into a learning moment; and (iv) private — it can run against a local model so that schema and data never leave the machine.

1.3 Problem Statement

To design and implement a secure, multi-database, AI-powered explorer that accepts a natural-language request, generates correct dialect-specific SQL (or a NoSQL command) using a Large Language Model with full schema context, validates the statement for safety and authorization,

executes read operations directly while routing write and schema operations through a human-in-the-loop review, and presents the results together with AI-generated analysis, visualizations, and exportable reports — all through a clean, responsive interface.

1.4 Objectives

1. To convert natural-language questions into validated, dialect-aware SQL across multiple database engines using an LLM with a schema-aware prompt.
2. To guarantee determinism and reliability for common schema-introspection tasks by answering them without invoking the LLM.
3. To enforce a two-layer safety model — query classification plus a destructive-action firewall — combined with role-based access control (Viewer, Editor, Admin).
4. To keep a human in the loop for all write and schema-altering operations, with dry-run preview and automatic snapshot/rollback.
5. To provide AI-assisted data analysis, automatic chart-type selection, auto-generated dashboards, and CSV/PowerPoint export.
6. To support both a privacy-preserving local model (Ollama) and a low-latency cloud model (Groq), with automatic failover between them.

1.5 Methodology

The methodology followed an incremental, layered approach. The pipeline that processes a user request is summarized in Fig. 1.1 and consists of the following stages:

A. Intent classification: Each request is first matched against a set of deterministic introspection commands; if matched, it is answered directly from the adapter layer with no model call. Otherwise it is treated as a data question.

B. Schema-aware generation: For data questions, the active adapter produces a rich schema description (columns, primary/foreign keys, indexes, and three live sample rows). This is injected into a dialect-specific prompt and sent to the LLM, which returns SQL and a plain-English explanation.

C. Validation and authorization: The generated statement is classified (READ / WRITE / SCHEMA / SYSTEM) and passed through a destructive-action safety filter, then checked against the user's role.

D. Execution or review: READ and SYSTEM statements execute immediately with pagination. WRITE and SCHEMA statements are stored and rendered on a review page; the user may edit, dry-run, or confirm them, after which an automatic snapshot is taken before execution.

E. Analysis and presentation: Results can be analyzed by the AI engine, which produces a textual summary and selects an appropriate chart, and can be exported as CSV or a formatted PowerPoint deck.

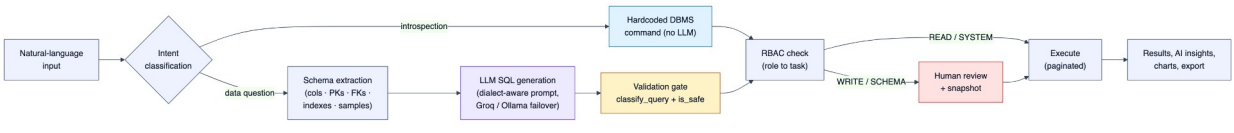


Fig. 1.1: Methodology pipeline of Meridian Data

1.6 Innovation

The innovation of Meridian Data lies less in a new parsing algorithm and more in the systems integration of components that prior work pursued in isolation. Specifically: (i) a hybrid execution model that combines deterministic, model-free introspection with LLM generation, giving correctness where it is cheap and intelligence where it is needed; (ii) a single adapter abstraction that extends natural-language querying uniformly across eight relational and NoSQL engines, including a JSON-command contract for MongoDB and Redis; (iii) automatic cloud-to-local provider failover that makes the tool resilient and privacy-flexible; and (iv) a defence-in-depth safety design in which the LLM is never trusted to be safe — a separate validator and an explicit human gate stand between generation and execution.

1.7 Organization of the Report

Chapter 1 introduces the problem, motivation, objectives, methodology, and innovation of the project.

Chapter 2 reviews the literature on natural-language interfaces, neural and LLM-based text-to-SQL, database safety, and automated visualization, and positions Meridian Data among them.

Chapter 3 specifies the functional, non-functional, hardware, and software requirements of the system.

Chapter 4 presents the module design and architecture — the high-level architecture, use-case and data-flow diagrams, structure chart, and a functional description of each module.

Chapter 5 describes the implementation — the sample datasets, the choice of programming language and platform, the libraries used, and representative code and pseudocode.

Chapter 6 reports results and discussion — evaluation metrics, experimental results with screenshots, unit/integration/system testing, and performance analysis on real telemetry.

Chapter 7 summarizes the work, draws conclusions, and outlines future enhancements.

Chapter 2: Literature Review

This chapter surveys the research that underpins Meridian Data, tracing the field from early natural-language interfaces to databases (NLIDBs) through neural text-to-SQL and the current LLM-based paradigm, and into the adjacent areas of database safety, access control, human oversight, and automated visualization. Figure 2.1 sketches this evolution. All sources are cited in IEEE style and listed in the References section.

Fig. 2.1: Evolution of text-to-SQL / NLIDB approaches

2.1 Literature Review

The relational model introduced by Codd [1] established the table-based foundation on which SQL and all modern relational database systems are built. While SQL is expressive and precise, it imposes a steep learning curve on non-technical users — the barrier that natural-language interfaces have sought to remove for decades. The seminal survey by Androutsopoulos, Ritchie and Thanisch [2] catalogued the early history of NLIDBs, from pattern-matching to semantic-grammar systems, and articulated the trade-offs of portability and linguistic coverage that still shape the field. Meridian Data inherits the central modern ambition — cross-domain generalization — by introspecting the schema of arbitrary databases at runtime rather than being trained on any single one.

The deep-learning era was catalysed by two datasets. Zhong, Xiong and Socher introduced WikiSQL with Seq2SQL [3], using reinforcement learning so that execution results, not just token loss, could supervise generation. Xu, Liu and Song's SQLNet [4] then showed that a sketch-based, sequence-to-set formulation with column attention could outperform Seq2SQL without reinforcement learning. Both were limited to single-table queries. Yu et al.'s Spider [5] reset the bar with 10,181 questions over 200 multi-domain, multi-table databases and a database-disjoint split, exposing how poorly prior models generalized. The architectural responses — RAT-SQL [6], with relation-aware self-attention unifying schema encoding and linking, and BRIDGE [7], which interleaves question tokens with schema and cell values — addressed schema linking and value grounding, two problems Meridian Data must also solve when binding a user's phrasing to concrete tables and columns. PICARD [8] contributed constrained autoregressive decoding that rejects syntactically invalid SQL at generation time — a guardrail philosophy that Meridian Data echoes in its post-generation validator. The survey of Katsogiannis-Meimarakis and Koutrika [9] provides the taxonomy that situates these systems.

The arrival of capable LLMs shifted the dominant paradigm from task-specific fine-tuning to in-context learning. Rajkumar, Li and Bahdanau [10] established that a general-purpose code LLM is a strong zero-/few-shot baseline on Spider. Pourreza and Rafiei's DIN-SQL [11] decomposed the task into schema linking, classification, generation, and self-correction, raising accuracy by roughly ten points; Gao et al.'s DAIL-SQL [12] systematically optimized prompt engineering while improving token efficiency. Li et al.'s BIRD benchmark [13] introduced large, "dirty" databases requiring external knowledge, where even GPT-4 reached only about

55% execution accuracy versus roughly 93% for humans — a reminder that human oversight remains essential. The survey by Hong et al. [14] consolidates LLM-based text-to-SQL. Meridian Data operationalizes these findings using an instruction-tuned open model (Llama 3.3 70B [15]) with schema-aware prompting, served either by Groq's deterministic Tensor Streaming Processor / LPU architecture [16] for low-latency inference or by a local Ollama runtime for privacy.

Because a generated query executes against a live database, correctness alone is insufficient — the system must be safe. The classic taxonomy of SQL-injection attacks and countermeasures by Halfond, Viegas and Orso [17] frames the threat model and motivates query whitelisting and the rejection of statement-stacking and comment markers, both of which Meridian Data enforces. Authorization is layered on top via Role-Based Access Control, whose reference models were formalized by Sandhu et al. [18]; roles constrain which operations the user, or the LLM acting on their behalf, may perform. Because LLM outputs are probabilistic, Meridian Data keeps a human in the loop: the survey by Wu et al. [19] on human-in-the-loop machine learning underpins the decision to surface generated SQL for inspection and confirmation before any write or expensive read.

Beyond returning rows, Meridian Data performs AI-assisted analysis and generates dashboards. This connects it to automated-visualization research: Dibia and Demiralp's Data2Vis [20] treats visualization design as neural sequence-to-sequence translation, while Narechania, Srinivasan and Stasko's NL4DV [21] maps natural-language queries to analytic tasks and recommended chart specifications. Meridian Data unifies these threads — LLM-based schema-aware generation [11], [12], [14], PICARD-style validation [8], RBAC authorization [18], injection-aware safety [17], explicit human oversight [19], and NL-driven visualization [20], [21] — in a single multi-database explorer. Its distinguishing contribution is therefore a systems integration of academically validated components rather than a new parsing algorithm.

Table 2.1: Summary of selected papers

Authors (Year) [Ref]	Objectives	Strengths & Weaknesses	Technique	Limitations
Zhong et al. (2017) [3]	NL→SQL over Wikipedia tables; release WikiSQL	First large dataset; RL handles order-invariant clauses; only single-table	Pointer network + policy-gradient RL (Seq2SQL)	No joins, nesting or cross-table aggregation
Xu et al. (2017) [4]	Generate SQL on WikiSQL without RL	Beats Seq2SQL by 9 –13% without RL; sketch hand-designed	Sketch-based slot filling + column attention (SQLNet)	Tied to WikiSQL's narrow grammar
Yu et al. (2018) [5]	Complex cross-domain text-to-SQL benchmark	200 multi-table DBs; forces generalization; schema-only	Human-labeled dataset; exact-set-match eval (Spider)	Best 2018 model only 12.4% exact match
Wang et al. (2020) [6]	Joint schema encoding and linking	+8.7% on Spider; heavy, fine-tuning-dependent	Relation-aware Transformer + grammar decoder	Compute-intensive; no zero-shot ability

Authors (Year) [Ref]	Objectives	Strengths & Weaknesses	Technique	Limitations
			(RAT-SQL)	
Scholak et al. (2021) [8]	Stop LMs emitting invalid SQL	Incremental parsing rejects bad tokens; needs grammar checker	Constrained autoregressive decoding (PICARD)	Ensures validity, not semantic correctness
Pourreza & Rafiei (2023) [11]	Improve LLM text-to-SQL by decomposition	~10% few-shot gain; no fine-tuning; token-costly	In-context decomposition + self-correction (DIN-SQL)	Latency/cost of multiple LLM calls
Li et al. (2023) [13]	Benchmark on large, dirty databases	Realistic; exposes model-vs-human gap	DB-grounded benchmark + valid-efficiency score (BIRD)	GPT-4 only ~55% vs ~93% human

Chapter 3: Requirements

This chapter specifies the requirements engineered for Meridian Data, separated into functional requirements (what the system must do), non-functional requirements (qualities it must exhibit), and the hardware and software needed to build and run it.

3.1 Functional Requirements

- The system shall authenticate users and assign one of three roles — Viewer, Editor, or Admin — governing the operations they may perform.
- The system shall accept a natural-language command and generate a corresponding dialect-specific SQL statement (or NoSQL command) using an LLM supplied with the live database schema.
- The system shall answer common introspection commands (show tables, describe, show foreign keys, show indexes, show constraints, show table counts, show create table) deterministically without invoking the LLM.
- The system shall classify every generated statement as READ, WRITE, SCHEMA, or SYSTEM and validate it against a destructive-action safety filter before execution.
- The system shall execute READ and SYSTEM statements immediately with pagination, and route WRITE and SCHEMA statements to a human review page supporting edit, dry-run, and confirmation.
- The system shall take an automatic snapshot before any write or schema change and provide one-click undo/restore.
- The system shall connect to multiple database engines (SQLite, PostgreSQL, MySQL, MSSQL, Oracle, MongoDB, Cassandra, Redis) and allow the user to switch the active connection.
- The system shall analyze a table, a custom query result, or an uploaded CSV with AI, producing a textual summary and an automatically selected chart.
- The system shall generate and persist multi-widget dashboards and export query results as CSV or a PowerPoint presentation.
- The system shall allow administrators to switch the active LLM provider, pull local models, and view usage telemetry.

3.2 Non-Functional Requirements

- **Security:** destructive DDL (DROP, TRUNCATE, ALTER), statement-stacking, and comment-based injection vectors must be rejected; credentials must be stored encrypted.
- **Reliability:** schema introspection must always succeed regardless of model availability; the system must fail over automatically from the cloud model to the local model.
- **Performance:** deterministic commands should respond in well under one second; LLM generation should typically complete within a few seconds on the cloud backend.

- **Usability:** the interface must be responsive, render Markdown answers, and explain the generated SQL so that it doubles as a learning aid.
- **Privacy:** the system must offer a fully local mode in which schema and data never leave the host.
- **Maintainability and Extensibility:** adding a new database engine must require only a new adapter implementing the common interface, with no change to the application logic.

3.3 Hardware Requirements

Meridian Data is lightweight on the client side (any modern browser) and modest on the server side. For the cloud-model configuration a basic development machine suffices; the local-model configuration benefits from more memory and, ideally, a GPU.

Table 3.1: Hardware requirements

Component	Minimum	Recommended
Processor	Dual-core x86-64 / Apple Silicon	Quad-core or better
RAM	4 GB (cloud model)	16 GB (to run a local Ollama model)
Storage	2 GB free	10 GB free (local models + snapshots)
GPU	Not required (cloud model)	Optional GPU for faster local inference
Network	Required for Groq cloud backend	Broadband for cloud; none for local mode

3.4 Software Requirements

The application is built on the Python/Flask ecosystem with a vanilla-JS Jinja interface and an optional React single-page application; AI is provided by the Groq SDK and an optional local Ollama runtime.

Table 3.2: Software requirements

Category	Requirement
Operating System	Windows, macOS, or Linux
Runtime	Python 3.11+ ; Node.js 20+ (for the React SPA build)
Backend framework	Flask 3.1 with flask-cors and python-dotenv
AI / LLM	Groq SDK (Llama 3.3 70B) ; optional Ollama (Mistral) for local inference
Database drivers	psycopg2, PyMySQL, pymssql, oracledb, pymongo, cassandra-driver, redis
Security / utilities	cryptography (Fernet), python-pptx, requests, Faker
Frontend	HTML5/CSS3, Chart.js, Marked.js, Mermaid ; React 19 + Vite + Tailwind CSS v4
Containerization	Docker and Docker Compose (optional one-command launch)

3.5 Summary

The requirements establish Meridian Data as a secure, reliable, multi-engine, privacy-flexible system whose functional scope spans natural-language querying, deterministic introspection, layered safety, human-in-the-loop writes, AI analysis, dashboards, and export. The next chapter translates these requirements into a concrete module design and architecture.

Chapter 4: Module Design & Architecture

This chapter presents the design of Meridian Data, beginning with a high-level architectural overview and the system's external interactions, followed by the detailed design — the structure chart and a functional description of each module.

4.1 Design Description

A high-level design illustrates the overall system architecture, presenting a wider picture of the aim and functionality of the different modules. Meridian Data follows a layered architecture with clear separation of concerns: a presentation layer (two interchangeable front-ends), an application layer (the Flask server hosting the query, analysis, dashboard, export, and snapshot engines), an LLM-provider layer with cloud-to-local failover, a data-access layer built on the Adapter design pattern, and a persistence layer of databases and JSON state files. The design deliberately isolates each engine behind a narrow interface so that it can be reasoned about and tested independently.

4.2 High Level Design

4.2.1 System Architecture

Figure 4.1 shows the high-level architecture. User requests from either front-end enter the Flask application through the authentication and RBAC gate. The Query Engine routes a request either to the deterministic hardcoded-command handler or to the schema-aware LLM router; generated statements pass through the validator before being executed by the appropriate database adapter or routed to the human-in-the-loop review. The Analysis, Dashboard, Export, and Snapshot engines reuse the same adapter layer, and all configuration and telemetry are persisted as JSON files.

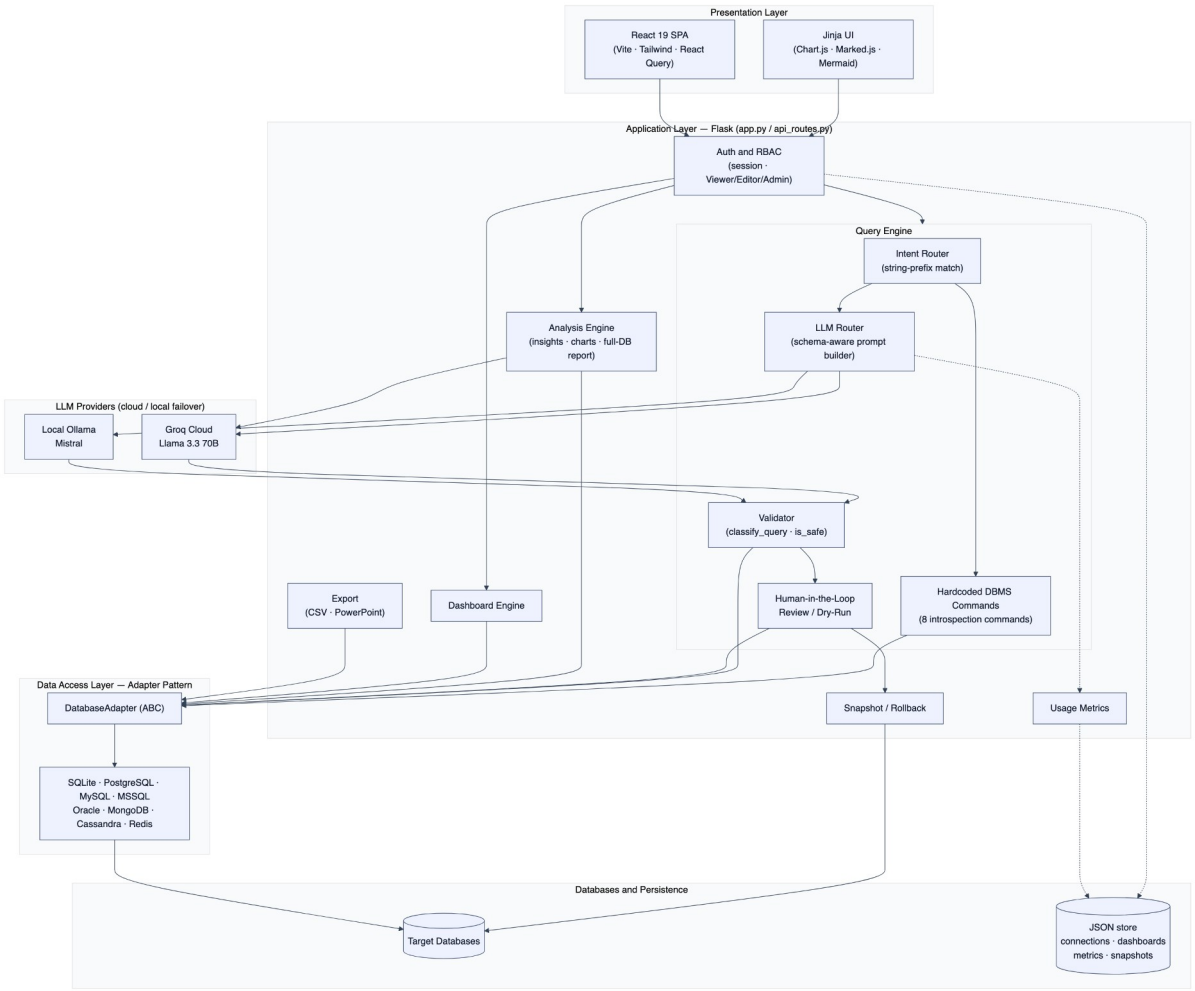


Fig. 4.1: High-level system architecture

The external interactions of the system are captured by the use-case diagram in Figure 4.2. Three cumulative roles are modelled — a Viewer (read and analyze), an Editor (additionally execute reviewed writes), and an Admin (additionally run schema changes, manage connections and providers, and view telemetry) — together with the two LLM providers that the system consults.

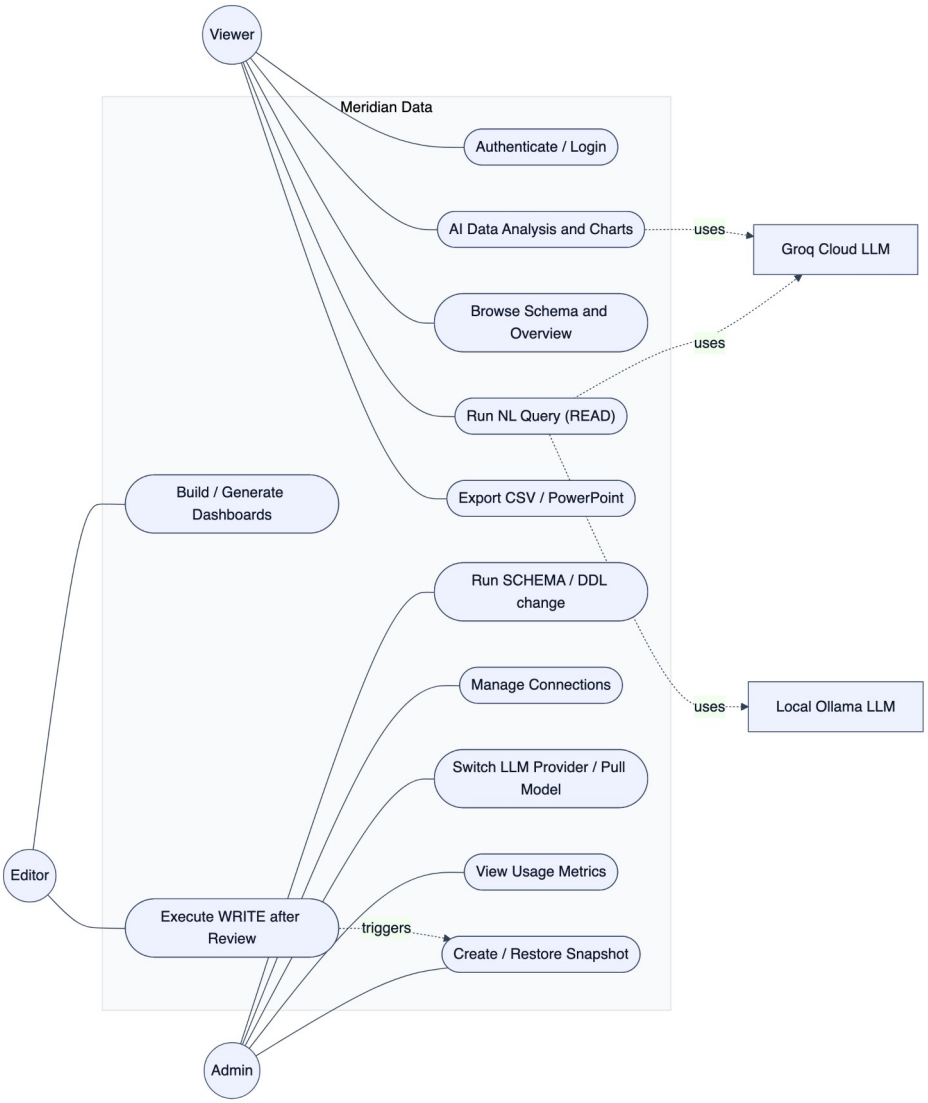


Fig. 4.2: Use-case diagram

Figures 4.3 and 4.4 present the data-flow view. The context diagram (Level 0) treats Meridian Data as a single process exchanging commands and results with the user, prompts and generated SQL with the LLM provider, and queries and rows with the target databases. The Level-1 diagram decomposes this into the authenticate, route-intent, generate, validate, execute/review, and analyze processes, together with the persistent stores for connections, conversation context, snapshots, and metrics.

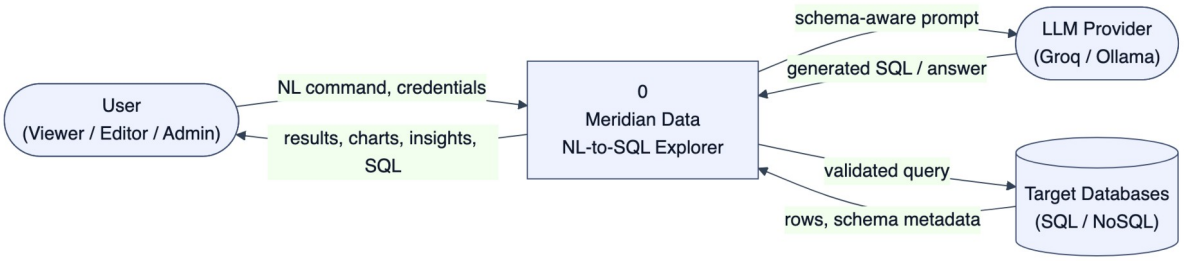


Fig. 4.3: Data Flow Diagram – Level 0 (context)

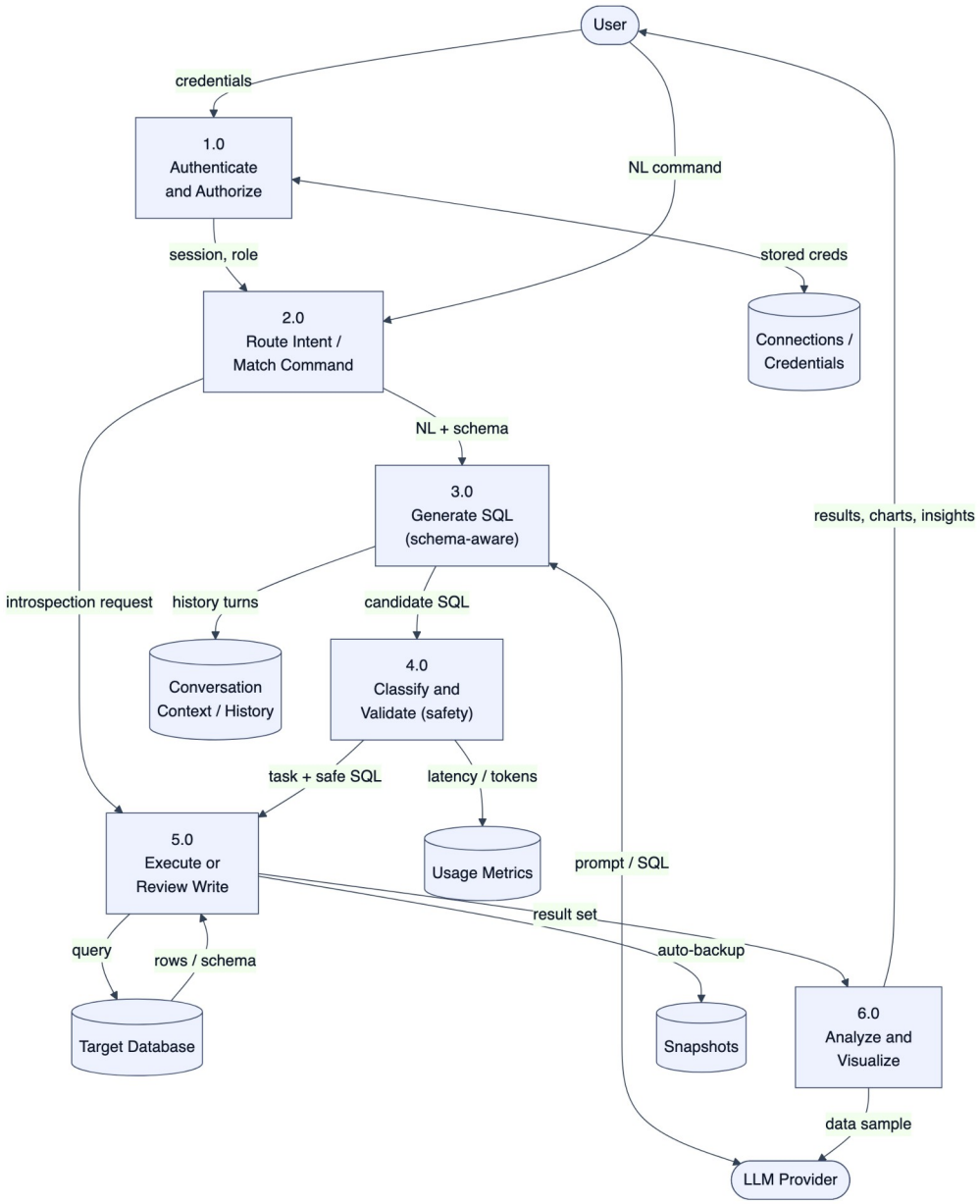


Fig. 4.4: Data Flow Diagram – Level 1

The dynamic behaviour of the central query lifecycle is shown by the sequence diagram in Figure 4.5, distinguishing the deterministic introspection path from the LLM path, and within the latter the READ, WRITE/SCHEMA-review, and blocked outcomes.

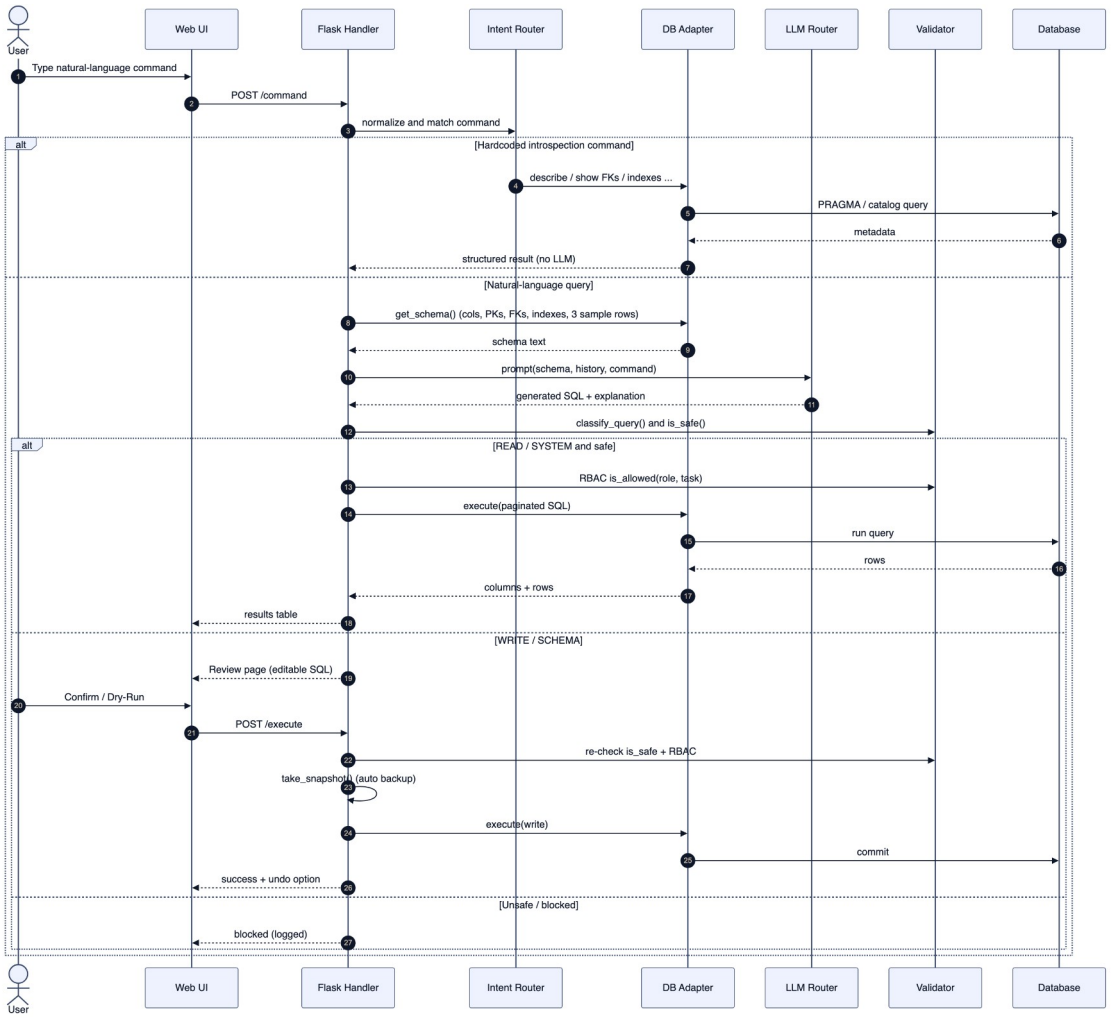


Fig. 4.5: Sequence diagram of the query lifecycle

4.3 Detailed Design

The detailed design refines the architecture into a hierarchy of modules and their responsibilities.

4.3.1 Structure Chart

A structure chart is a top-down, hierarchical diagram that visualizes a system's architecture by breaking it into modules and representing their dependencies. Figure 4.6 shows the decomposition of Meridian Data into its principal modules and sub-modules.

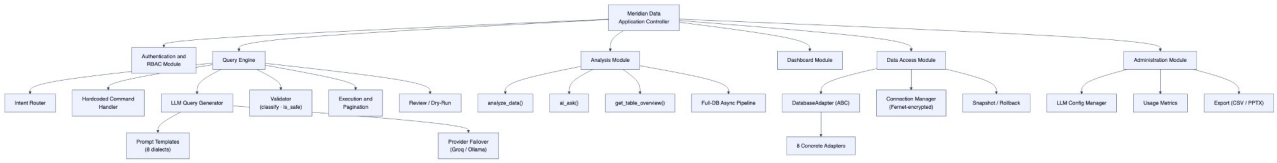


Fig. 4.6: Structure chart of modules

The application controller dispatches to six top-level modules. The Query Engine is decomposed into the intent router, the hardcoded command handler, the LLM query generator (itself comprising the eight dialect prompt templates and the provider-failover logic), the validator, the execution/pagination component, and the review/dry-run component. The Data

Access module centres on the abstract adapter and its eight concrete implementations, the Fernet-based connection manager, and the snapshot engine.

4.3.2 Functional Description of the Modules

1. Authentication and RBAC Module. Maintains session state, verifies credentials against Werkzeug-hashed demo accounts, and enforces a permission matrix in which Viewer holds {READ, SYSTEM}, Editor adds {WRITE}, and Admin adds {SCHEMA}. A before-request guard protects every route, returning JSON 401 for API paths and redirecting browser paths to the login page.

2. Query Engine. The heart of the system. It normalizes the input, matches it against eight deterministic introspection commands, and otherwise builds a schema-aware prompt and calls the LLM router. The generated statement is classified and safety-checked, then either executed with pagination (LIMIT/OFFSET of 50) or routed to review.

3. LLM Router. Selects one of eight dialect-specific prompt templates, injects the schema (columns, primary keys, foreign keys, indexes, and three sample rows) and the last five conversation turns, and issues the request along a provider chain that fails over between Groq (cloud Llama 3.3 70B) and Ollama (local Mistral). It strips code fences from the output and logs latency and token counts.

4. Validator Module. Implements classify_query (READ/WRITE/SCHEMA/SYSTEM/UNKNOWN) and is_safe, the destructive-action firewall that rejects DROP, TRUNCATE, ALTER, ATTACH/DETACH, PRAGMA, SHUTDOWN, comment markers, statement-stacking, and the NoSQL equivalents (dropDatabase, FLUSHALL, CONFIG).

5. Data Access Module. Realizes the Adapter pattern: an abstract DatabaseAdapter defines connect, disconnect, test_connection, get_schema, list_tables, and execute, plus introspection and snapshot hooks. Eight concrete adapters translate these to engine-native calls; pooling is provided where cheap (PostgreSQL native pool, MySQL queue pool, SQLite singleton).

6. Analysis Module. Orchestrates Groq calls for single-dataset analysis (≤100 rows, JSON-mode, six chart types), conversational Q&A with full schema context, a database overview, and a two-call asynchronous full-database report executed on a background thread with per-query timeouts.

7. Dashboard Module. A JSON-backed store of dashboards and widgets, where each widget is a saved query plus a chart type; data is re-fetched live on every view so dashboards always reflect current state.

8. Snapshot, Export, and Administration Modules. The snapshot engine takes per-engine backups (file copy, pg_dump, mysqldump, mongodump) with five-per-database retention; the export module produces CSV and a themed PowerPoint deck; the administration module persists provider configuration and exposes usage telemetry.

Chapter 5: Implementation

This chapter describes how the design of Chapter 4 was realized — the datasets used for development and demonstration, the choice of programming language and platform, the libraries and tools employed, and representative code and pseudocode for the system's central algorithms.

5.1 Dataset Description

Because Meridian Data is schema-agnostic, it ships with several well-known sample databases so that its behaviour can be demonstrated and tested without external setup. The default working database is a clone of the Chinook digital-media store (carrying one additional auxiliary test table, so the live application reports twelve tables for the default connection); the larger Northwind trading database is used to exercise the system on substantial data volumes. Table 5.1 summarizes the bundled datasets.

Table 5.1: Sample databases bundled with Meridian Data

Database	Engine	Tables	Largest table (rows)	Domain
Chinook (default)	SQLite	11	PlaylistTrack (8,715)	Digital media store
Northwind	SQLite	13 (+17 views)	Order Details (609,283)	Trading / orders
Hospital (synthetic)	SQLite	—	—	Healthcare (Faker-generated)
Sakila	SQLite	—	—	DVD rental
Kibi Leads	PostgreSQL	—	—	CRM (external connection)

The Chinook schema, used for most demonstrations in this report, comprises eleven tables linked by eleven foreign-key relationships, including a self-reference on Employee (the ReportsTo manager hierarchy) and a many-to-many junction (PlaylistTrack) between Playlist and Track. Its entity-relationship diagram is shown in Figure 5.1.

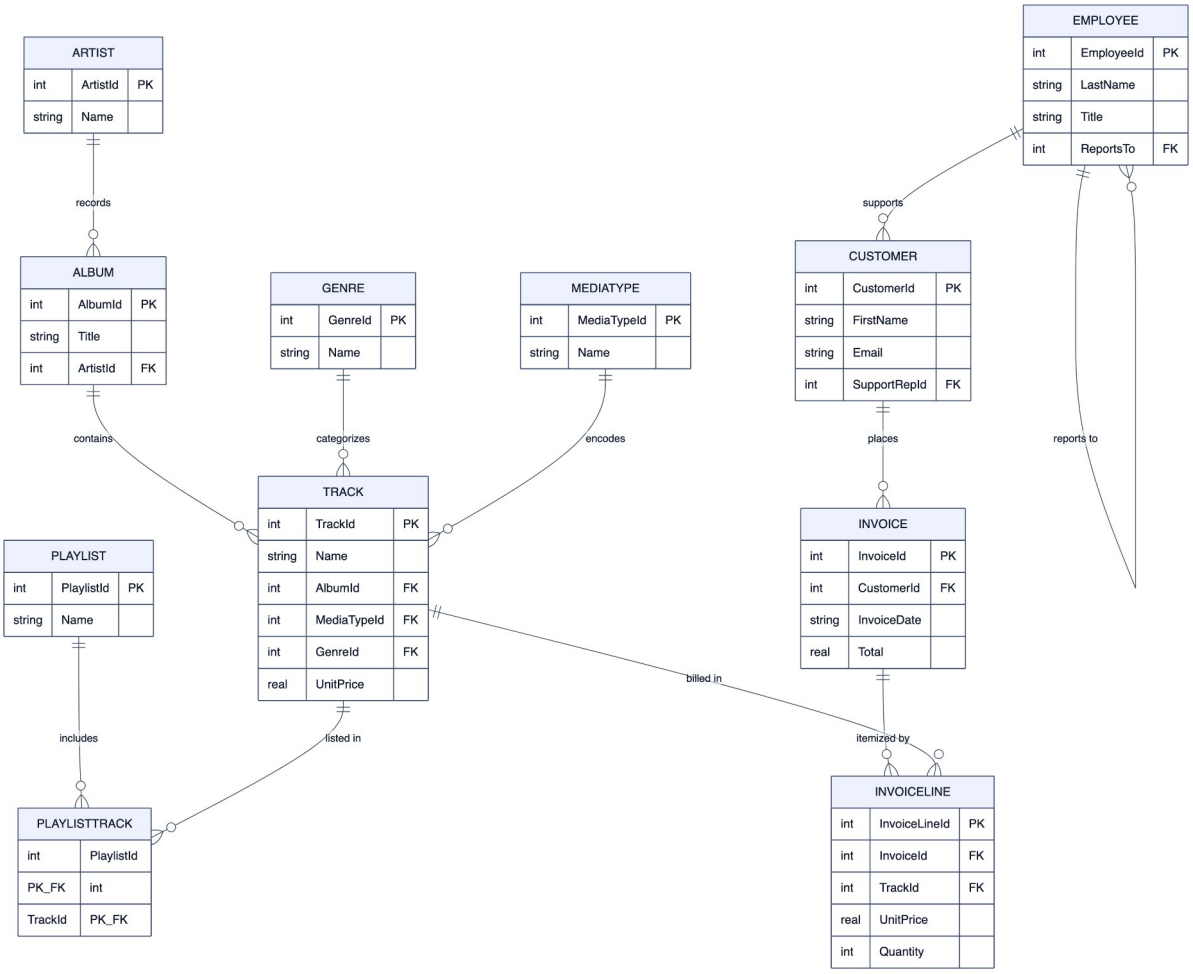


Fig. 5.1: ER diagram of the Chinook sample database

5.2 Programming Language Selection

Python 3.11 was selected as the primary implementation language for the backend. Python offers a mature data and AI ecosystem, first-class database drivers for every engine the project targets, and the Groq and Ollama client libraries, all of which minimized integration effort. Its readability also suits a project whose secondary goal is pedagogical clarity. The interactive front-ends are written in HTML5, CSS3, and JavaScript — a server-rendered Jinja interface using Chart.js, Marked.js, and Mermaid, and a modern React 19 single-page application. SQL (in eight dialect variants) and JSON command objects (for MongoDB and Redis) are the languages the system generates.

5.3 Platform Selection

The platform is the Flask web framework running on the CPython runtime, chosen for its minimalism and its suitability for a request-response architecture with server-side sessions. The application is cross-platform (Windows, macOS, Linux) and is additionally packaged with Docker and Docker Compose for one-command deployment. Cloud inference is delegated to Groq, whose Language Processing Unit hardware delivers low-latency generation, while privacy-sensitive deployments use a local Ollama runtime. The libraries and tools used are listed in Table 5.2.

Table 5.2: Key libraries and tools used

Purpose	Library / Tool
Web framework	Flask 3.1, flask-cors, python-dotenv
Cloud LLM	groq (Llama 3.3 70B Versatile)
Local LLM	Ollama (Mistral)
Database drivers	psycopg2-binary, PyMySQL, pymssql, oracledb, pymongo, cassandra-driver, redis
Security	cryptography (Fernet symmetric encryption), Werkzeug password hashing
Export	python-pptx (PowerPoint), Python csv module
Synthetic data	Faker
Frontend (Jinja)	Chart.js, Marked.js, Mermaid
Frontend (SPA)	React 19, Vite, Tailwind CSS v4, TanStack React Query, axios

5.4 Code Snippet / Pseudocode

The central dispatcher routes a command first through the deterministic introspection commands and only then to the LLM, after which it classifies, authorizes, and either executes or sends the statement to review. Its logic is summarized by the following pseudocode:

```
function handle_command(user_cmd, session):
    adapter = get_active_adapter(session); dialect = adapter.dialect
    cmd = lowercase(strip(user_cmd)); role = session.role
    # (1) Deterministic introspection — no LLM
    if cmd matches a hardcoded command (describe / show fks / indexes ...):
        return render(adapter.<introspection>())            # SYSTEM
    # (2) Schema-aware LLM generation
    schema  = adapter.get_schema()   # cols, PKs, FKs, indexes, 3 samples
    history = session.conversation_context[-5:]
    (sql, explanation) = generate_query_with_explanation(
                            user_cmd, dialect, schema, provider, history)
    # (3) Classify + safety + RBAC
    task = classify_query(sql, dialect)
    if task == UNKNOWN and role in {ADMIN,EDITOR} and is_safe(sql): task =
READ
    if not is_allowed(role, task): return blocked()
    # (4) Execute or review
    if task in {READ, SYSTEM}:
        if not is_safe(sql): return blocked()
        return execute_with_pagination(adapter, sql, page)
    else:   # WRITE / SCHEMA
        stash(sql, task); return review_page(sql, explanation)
```

Provider failover is expressed as an ordered chain so that the system degrades gracefully when one backend is unavailable:

```
function generate_query(cmd, dialect, schema, provider, history):
    context = system_prompt(dialect, schema)        # template + schema
    chain = (provider == 'mistral') ? [mistral, groq] : [groq, mistral]
    for (name, cfg) in chain:
        try:
            out = call(name, context, history, cmd, temperature=0.1)
            log_metrics(name, latency, tokens)
            return clean_sql(out)        # first success wins
        except e: continue               # fall through to next provider
    return 'ERROR: all providers failed'
```

The safety gate that stands between generation and execution rejects destructive and injection-style statements before any query reaches the database:

```
function is_safe(query, dialect):
    q = lower(strip(query))
    if dialect in {mongodb, redis}:
        if any NOSQL_DANGEROUS token in q: return False  # dropDatabase,
FLUSHALL ...
        return True
    for kw in {'drop ','truncate ','alter ','shutdown','attach ',
               'detach ','pragma ','--','/*','*/'}:
        if kw in q: return False
    if ';' in query: return False            # block statement stacking
    if q starts_with 'select': return True
    if q starts_with ('insert','update','delete'): return True
    if classify_query(query, dialect) == SYSTEM: return True
    return (no danger char in q)             # permissive text fallback
```

5.5 Summary

The implementation realizes the layered design on a Python/Flask platform with a hybrid deterministic-plus-LLM query engine, an eight-engine adapter layer, and a two-stage safety gate, demonstrated on standard sample databases. The next chapter evaluates the resulting system.

Chapter 6: Results and Discussion

This chapter evaluates Meridian Data against its objectives. It defines the evaluation metrics, presents experimental results with screenshots of the working system, reports unit, integration, and system testing, and analyses performance using real usage telemetry collected during development and testing.

6.1 Evaluation Metrics

Because Meridian Data is a systems-integration project rather than a new model, it is evaluated on operational metrics rather than on a leaderboard accuracy score. The metrics used are: (i) functional correctness — whether each feature produces the expected result on the sample databases; (ii) generation correctness — whether the SQL produced for a natural-language request executes and returns the intended rows; (iii) safety — whether destructive and unauthorized statements are blocked; (iv) latency — the wall-clock time to answer, separated by execution path and provider; and (v) token efficiency — prompt and completion tokens per call. The latency and token metrics are measured directly from the application's telemetry log.

6.2 Experimental Results

The system was exercised through both front-ends against the bundled databases. Access begins at the role-aware login screen (Figure 6.1), after which the user reaches the natural-language query interface (Figure 6.2) with its query history, active-database selector, and provider toggle.

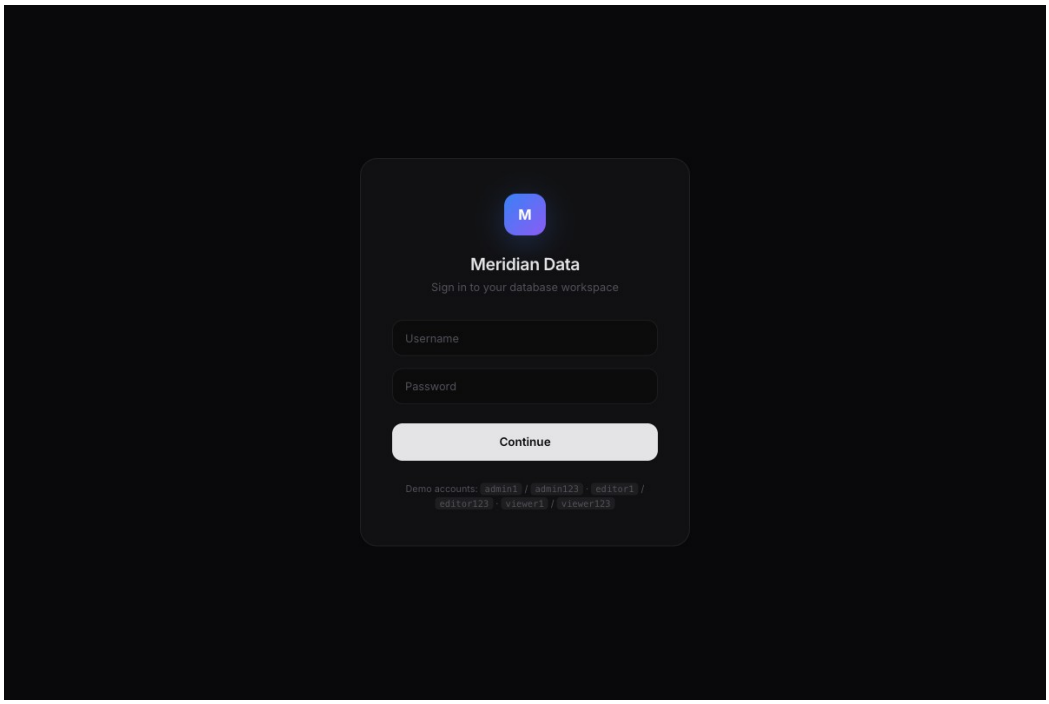


Fig. 6.1: Login screen (role-based access)

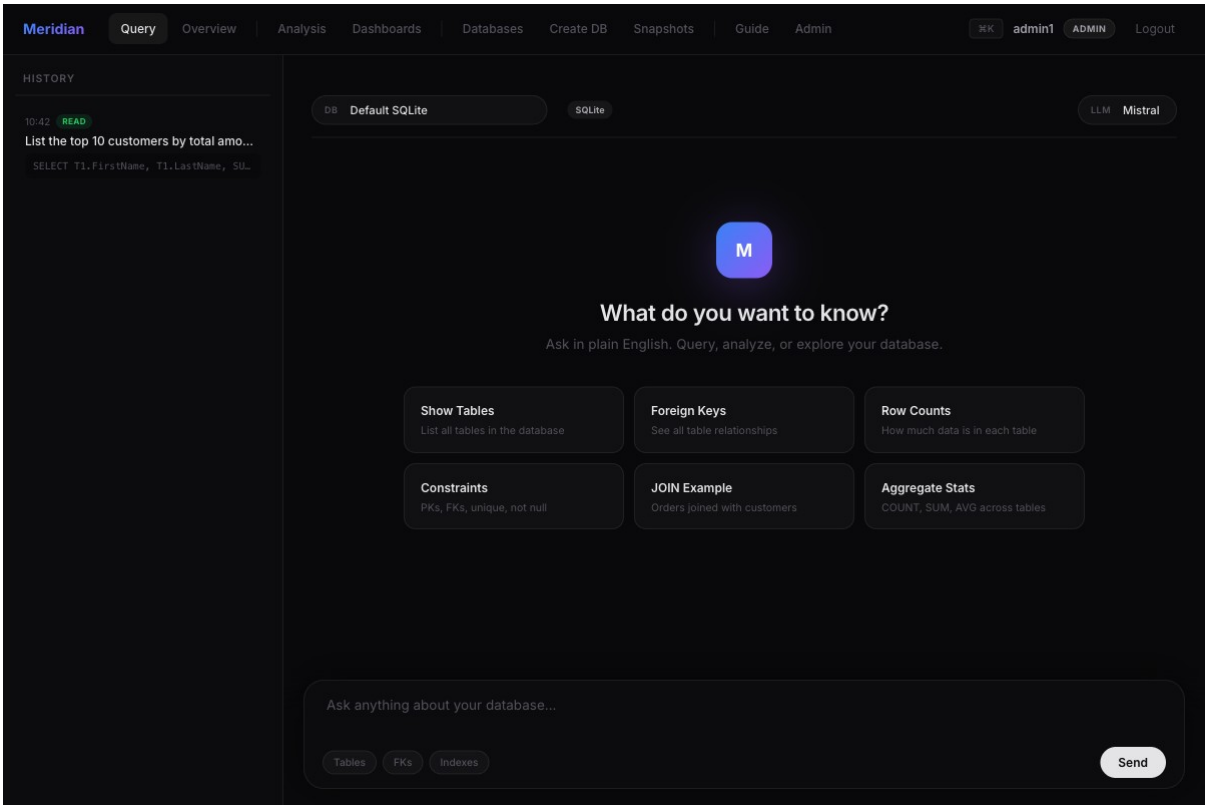


Fig. 6.2: Natural-language query interface

Entering a natural-language request such as “list the top 10 customers by total amount they have spent” produces a generated SQL statement together with a plain-English explanation; when a request is better answered conversationally, the assistant responds in Markdown with the relevant SQL and reasoning, as shown in Figure 6.3.

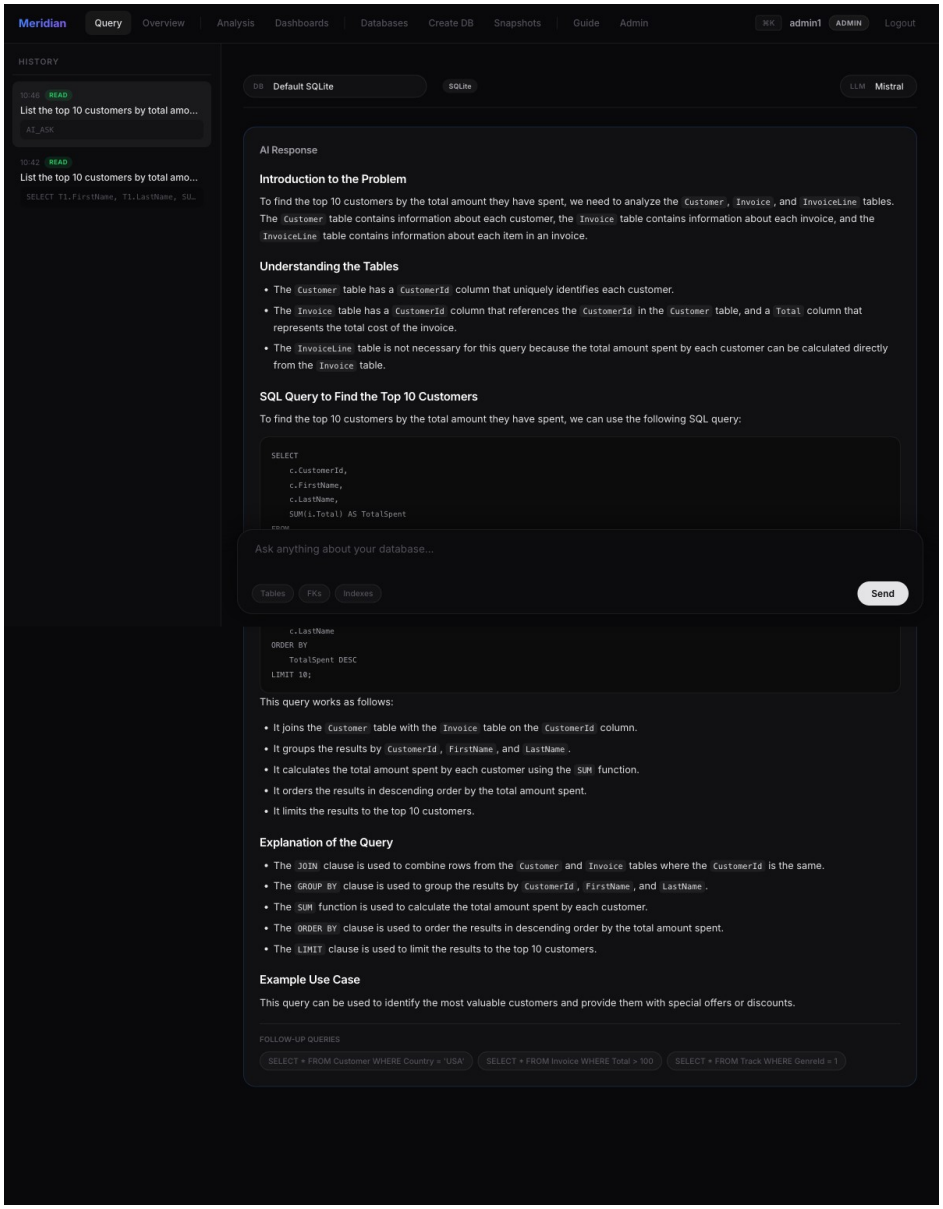


Fig. 6.3: AI response with generated SQL and explanation

The Database Overview page (Figure 6.4) demonstrates the deterministic and AI capabilities working together: it reports stat cards (12 tables — the eleven Chinook tables plus one auxiliary test table in the default database — 7,131 rows, 11 foreign keys, largest table PlaylistTrack with 8,715 rows), an AI executive summary, a table-size chart, the relationship list, and an automatically rendered entity-relationship diagram.

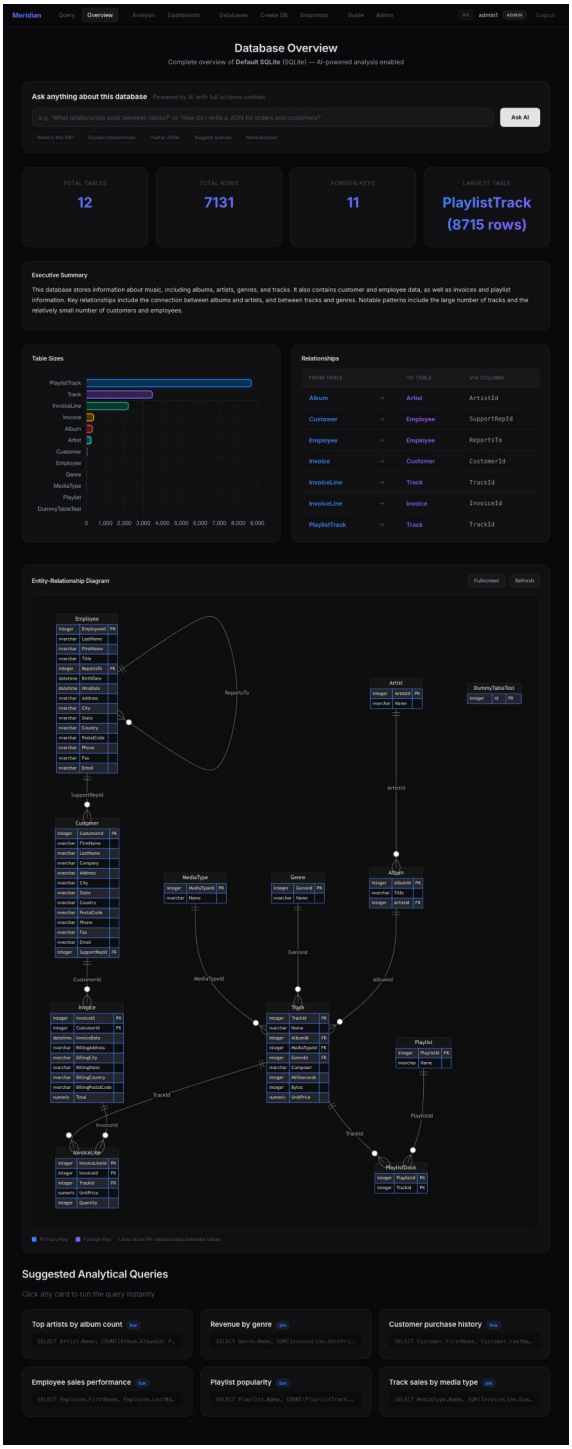


Fig. 6.4: Database Overview with auto-generated ER diagram

The AI Data Analysis workbench (Figure 6.5) lets the user pick a table, write a custom query, upload a CSV, or analyse the entire database; it returns a written summary and an automatically chosen chart — here a bar chart of invoices by country.

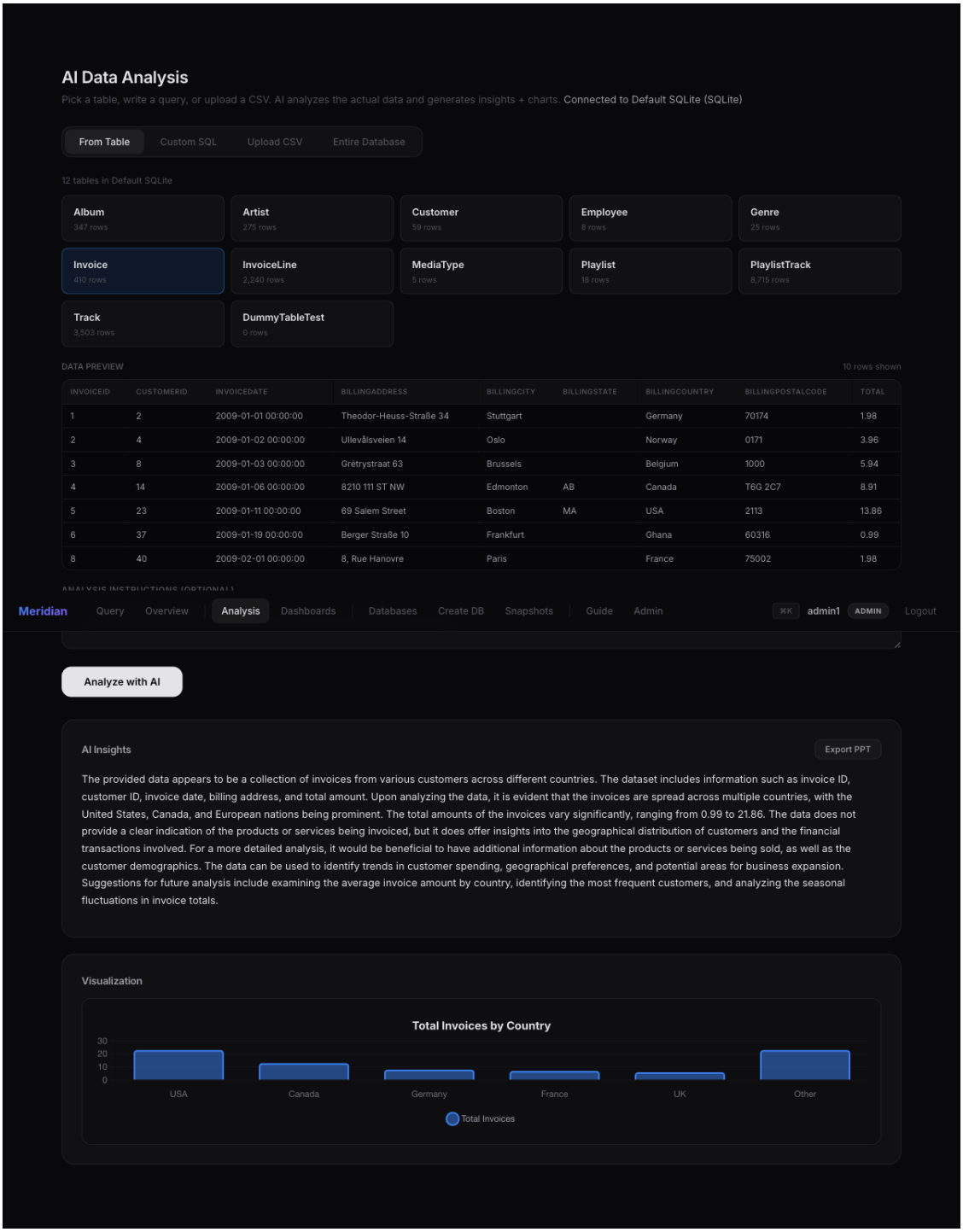


Fig. 6.5: AI Data Analysis with generated chart

Write operations are never executed silently. A request that adds a row produces the human-in-the-loop review page of Figure 6.6, which shows the editable INSERT statement, the AI explanation, a refine box, and the Execute / Dry-Run / Undo controls.

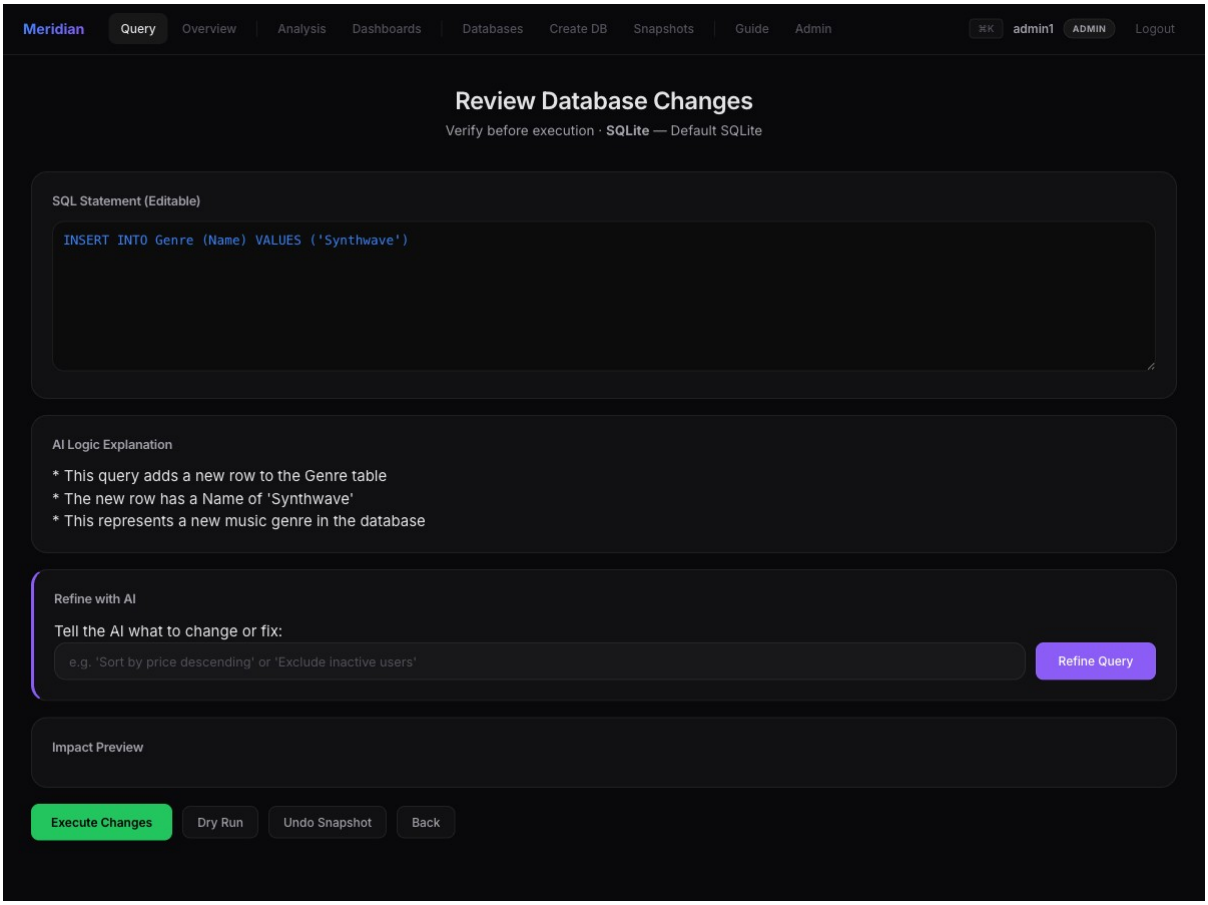


Fig. 6.6: Human-in-the-loop review page

Multi-database support is managed from the connection manager (Figure 6.7), which lists the saved SQLite and PostgreSQL connections and allows testing and switching. The dashboard module (Figure 6.8) renders multiple live widgets — here a bar chart, a pie chart, and a data table of customer distribution by country — each re-querying the database on load.

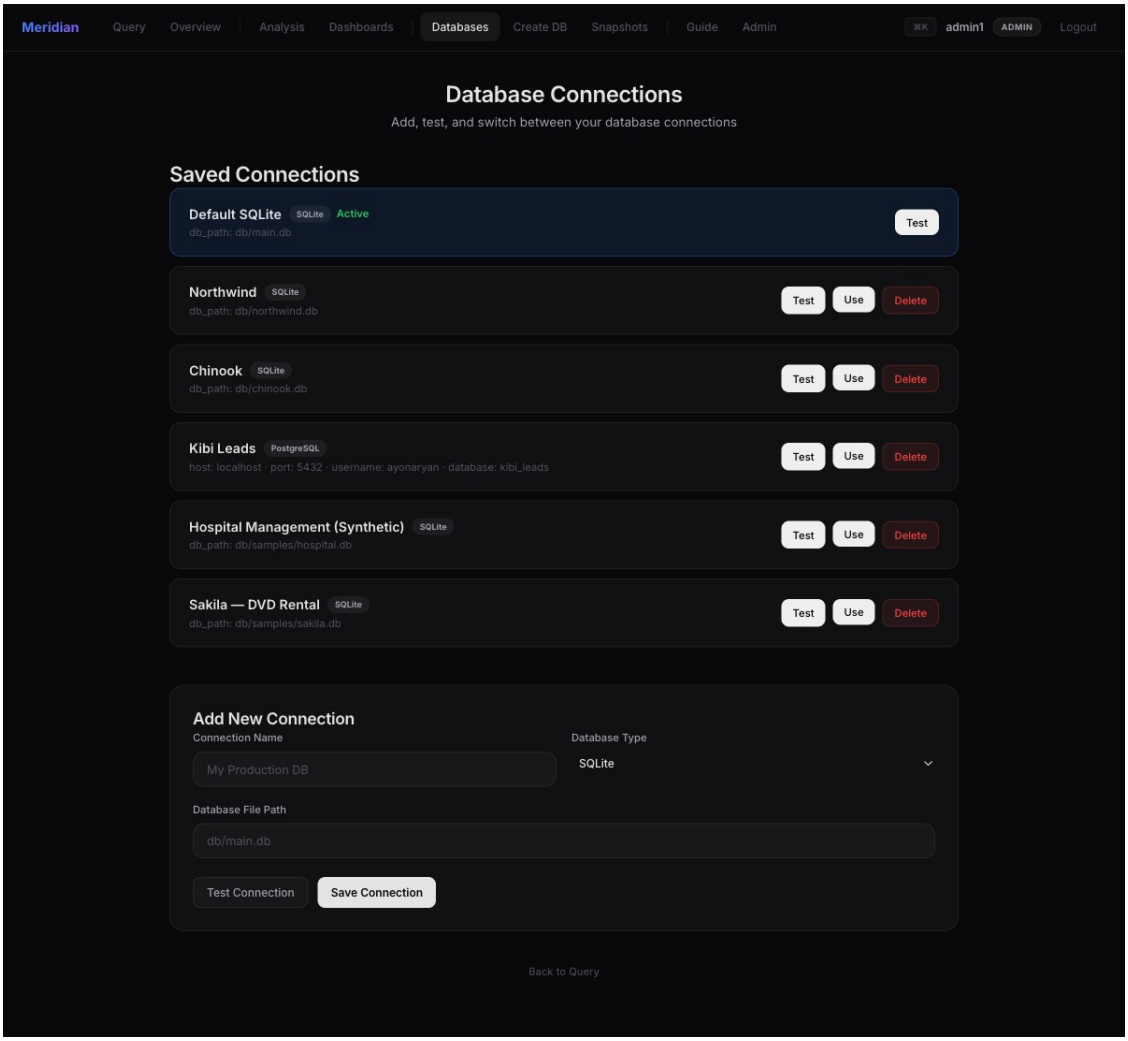
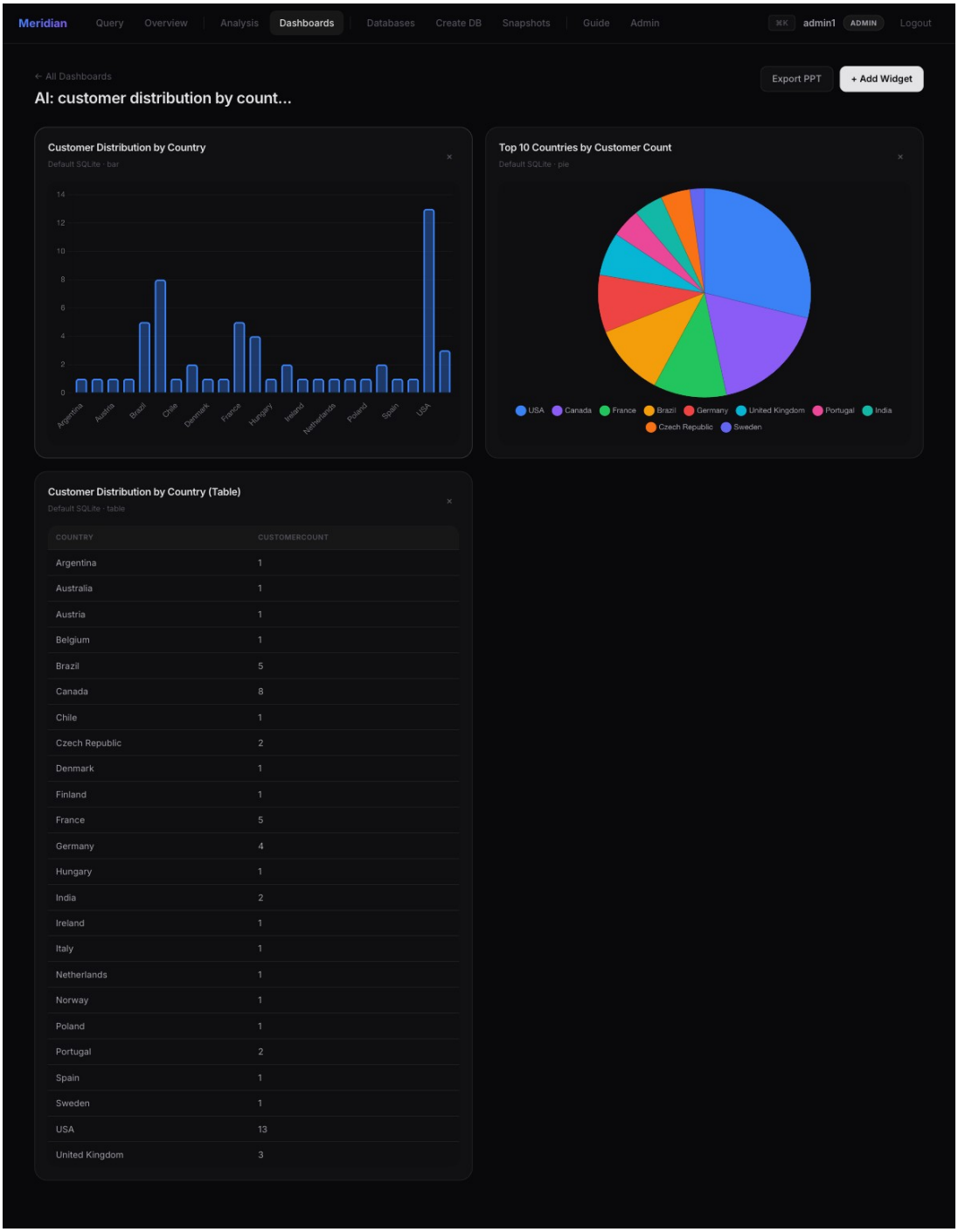


Fig. 6.7: Multi-database connection manager



COUNTRY	CUSTOMERCOUNT
Argentina	1
Australia	1
Austria	1
Belgium	1
Brazil	5
Canada	8
Chile	1
Czech Republic	2
Denmark	1
Finland	1
France	5
Germany	4
Hungary	1
India	2
Ireland	1
Italy	1
Netherlands	1
Norway	1
Poland	1
Portugal	2
Spain	1
Sweden	1
USA	13
United Kingdom	3

Fig. 6.8: AI-generated dashboard with live charts

Finally, the administration dashboard (Figure 6.9) visualizes usage telemetry — total API calls, average latency, total tokens, provider distribution, and token/latency trends — which forms the basis of the performance analysis in Section 6.4.

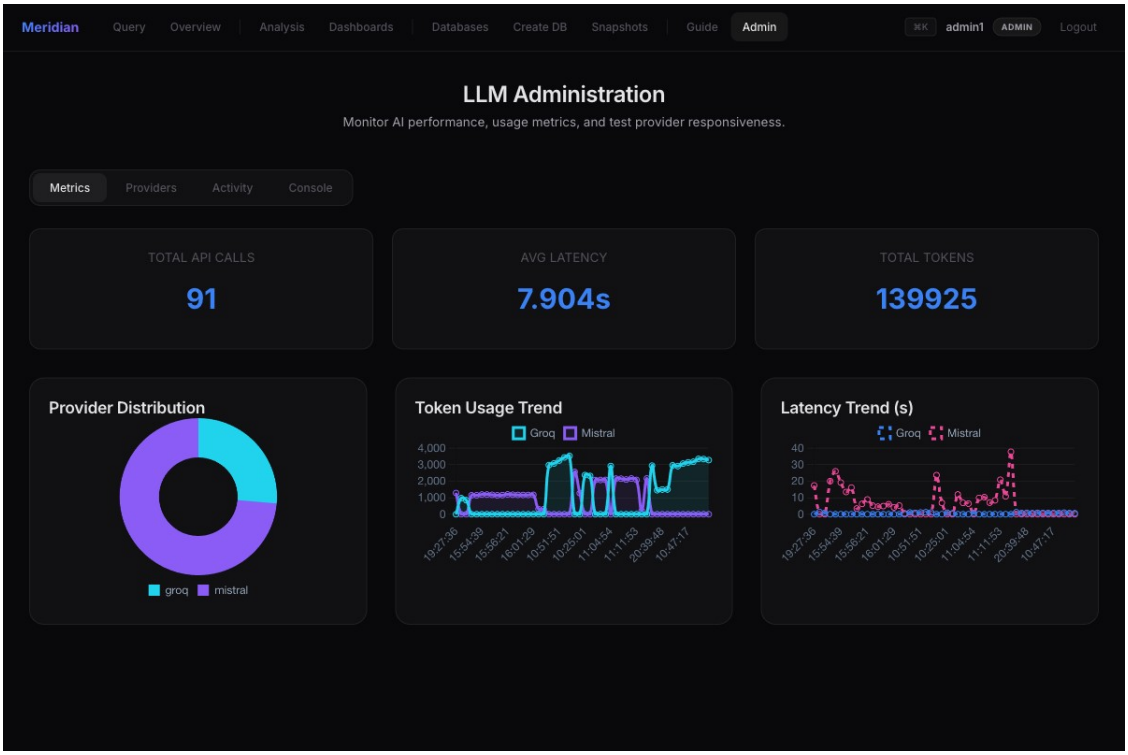


Fig. 6.9: LLM administration and usage telemetry

6.3 Validation and Testing

Validation was carried out at three levels — unit, integration, and system — using the live application connected to the default Chinook database. The inputs, expected outputs, and observed outputs were recorded directly from the running system on 25 May 2026.

6.3.1 Unit Testing

Unit testing exercises individual modules in isolation. The deterministic command handlers and the validator were tested first, since the remainder of the pipeline depends on them. All cases passed; introspection commands responded in under 20 ms.

Table 6.1: Unit testing results

No.	Module Tested	Input	Expected Output	Observed Output	Result
1	Hardcoded handler	show tables	List of all tables	12 tables (SYSTEM, 12 ms)	Pass
2	Hardcoded handler	describe Customer	Columns, keys, indexes	5 cols × 19 rows (4 ms)	Pass
3	Hardcoded handler	show foreign keys	All FK relationships	11 relationships (3 ms)	Pass
4	Validator (is_safe)	truncate table Invoice	Reject as unsafe	Blocked, not executed	Pass
5	Validator (classify)	SELECT ... FROM Customer	Classified READ	READ	Pass
6	LLM router	“invoices per country”	Valid SELECT +	Correct SQL; 25	Pass

No.	Module Tested	Input	Expected Output	Observed Output	Result
			rows	rows	

6.3.2 Integration Testing

Integration testing inspects how modules work together — in particular the generation→validation→authorization→execution chain and the write-review→snapshot→execute flow.

Table 6.2: Integration testing results

No.	Integration	Input	Expected Output	Observed Output	Result
1	LLM → Validator → Execute	NL read request	Generated SQL executes, paginated	Correct rows returned (~0.8 s)	Pass
2	Validator → RBAC → Review	"add a new genre Synthwave"	WRITE intercepted for review	INSERT shown on review page	Pass
3	Review → Snapshot → Execute	Confirm reviewed INSERT	Snapshot then commit	Backup taken, row inserted	Pass
4	Generator → Provider failover	Local model offline	Fail over to Groq	Cloud answered automatically	Pass
5	Adapter → Overview → ER render	Open Overview	Stats + ER diagram	Rendered with live data	Pass

6.3.3 System Testing

System testing evaluates the fully integrated application as a black box through the user interface.

Table 6.3: System testing results

No.	Scenario	Expected Output	Observed Output	Result
1	Login as each role	Role-appropriate access	Viewer/Editor/Admin gated correctly	Pass
2	End-to-end NL query	Result table + export options	Worked across both front-ends	Pass
3	Switch active database	New schema in context	Northwind/Chinook switched cleanly	Pass
4	AI analysis + chart	Summary and chart	Generated for selected table	Pass
5	Generate dashboard	Live multi-widget board	3 widgets rendered with live data	Pass
6	Destructive command	Blocked safely	DROP/TRUNCATE never executed	Pass

6.4 Performance Analysis

Performance was measured from the application's own telemetry log, which records the latency and token counts of every model call, supplemented by per-request timings captured during the functional tests. Across the recorded calls, the Groq cloud backend answered with a mean latency of 0.85 s (median 0.80 s, minimum 0.25 s), whereas the local Mistral model averaged 10.4 s on the same hardware — the expected trade-off between cloud acceleration and on-device privacy. Deterministic introspection commands, which bypass the model entirely, completed in roughly 8 ms. Table 6.4 and Figures 6.10–6.11 summarize these results.

Table 6.4: Performance parameters (measured)

Execution path	Mean latency	Median	Mean tokens/call	Notes
Hardcoded introspection	~8 ms	—	0	No LLM call; deterministic
NL→SQL (Groq, Llama 3.3 70B)	0.85 s	0.80 s	2,529	Cloud LPU inference
NL→SQL (local Mistral)	10.43 s	8.76 s	1,193	On-device, fully private

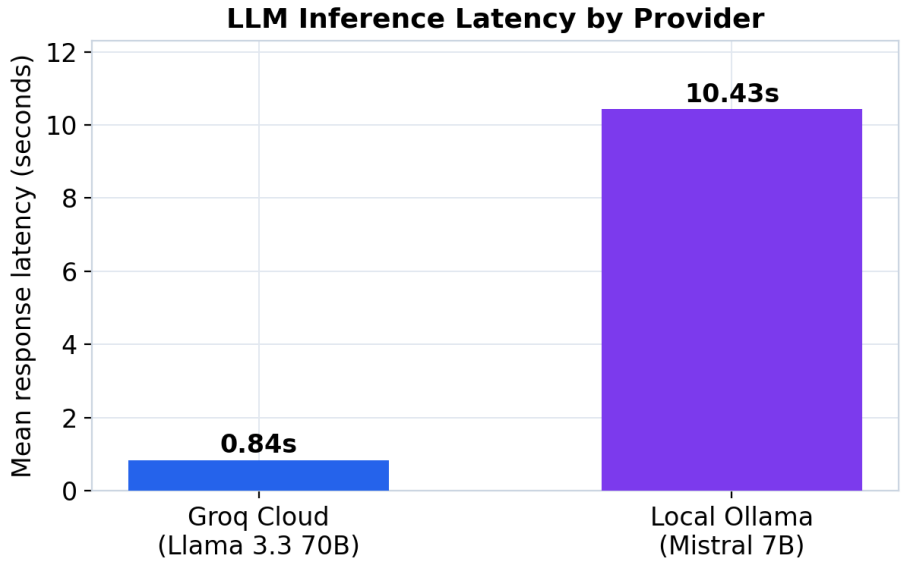


Fig. 6.10: Mean inference latency by provider

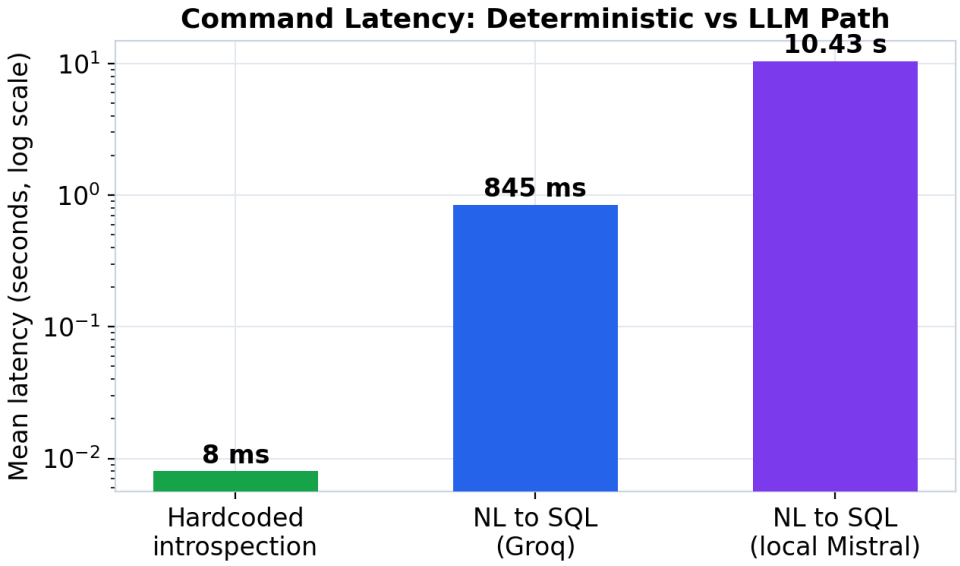


Fig. 6.11: Latency — deterministic vs LLM path (log scale)

Two design decisions are validated by these measurements. First, answering introspection commands deterministically yields a roughly hundred-fold latency improvement over a model call for the most frequent operations, while also guaranteeing correctness. Second, cloud-to-local failover lets the same system trade latency for privacy: a user who cannot send data to the cloud accepts a slower local model, while others enjoy sub-second cloud responses.

Observed limitation. During testing, the JSON API endpoint was observed to occasionally echo the previous read query's result set, because the response is built from a session field set on a prior request; the server-rendered interface, which renders within the same request, does not exhibit this. This and other limitations are discussed in Chapter 7.

6.5 Summary

Meridian Data met all of its functional objectives: it generated correct SQL from natural language, answered introspection deterministically, blocked every destructive statement, intercepted writes for human confirmation, analysed data with automatic charting, generated dashboards, and exported reports. Measured performance confirmed sub-second cloud generation and millisecond introspection, validating the hybrid execution and failover design.

Chapter 7: Summary, Conclusion & Future Enhancements

7.1 Summary

This project set out to remove the SQL barrier that stands between users and their data, without compromising on safety or control. We designed and implemented Meridian Data, a full-stack, AI-powered explorer that converts natural language into validated, dialect-specific queries across eight database engines. Chapter 1 framed the problem and objectives; Chapter 2 positioned the work within the text-to-SQL and database-safety literature; Chapter 3 specified the requirements; Chapter 4 presented the layered architecture and detailed module design; Chapter 5 described the implementation on a Python/Flask platform with a hybrid deterministic-plus-LLM query engine and an adapter-based data layer; and Chapter 6 evaluated the system, reporting correct generation, complete safety enforcement, sub-second cloud latency, and millisecond introspection on real telemetry.

The major findings are that a hybrid execution model (deterministic introspection plus LLM generation) delivers both reliability and intelligence; that a separate validator and an explicit human gate are effective at preventing the LLM from causing harm; that a single adapter abstraction can extend natural-language querying uniformly across relational and NoSQL engines; and that cloud-to-local failover makes the tool simultaneously fast and privacy-flexible.

7.2 Conclusion

Meridian Data demonstrates that the recent advances in LLM-based text-to-SQL can be turned into a dependable, everyday tool when they are wrapped in the right engineering: schema-aware prompting for accuracy, deterministic shortcuts for reliability, layered validation and role-based access control for safety, human-in-the-loop review for trust, and provider failover for resilience and privacy. The system fulfils its dual purpose as both a productivity aid for analysts and a teaching instrument for students, who can see, edit, and learn from the SQL it generates. In doing so it confirms the project's central thesis: that the value lies not in a new parsing algorithm but in the careful integration of academically validated components into a safe, usable whole.

7.3 Future Enhancements

- Replace the keyword-based safety filter with a full SQL parser (e.g. sqlglot) for parser-level validation and semantic checks, eliminating the heuristic's edge cases.
- Unify the duplicated server-rendered and JSON pipelines behind a single service layer to remove drift and the observed session-state echo on the API.
- Add retrieval-augmented schema selection so that very large schemas are summarized before prompting, improving accuracy and reducing token cost.
- Introduce automatic self-correction: feed execution errors back to the model (DIN-SQL style) to repair failed queries before falling back to a natural-language answer.
- Strengthen credential security by deriving the encryption key from an operator passphrase or a system keystore rather than an on-disk file.

- Extend evaluation with a quantitative accuracy study on the Spider and BIRD benchmarks, and add fine-grained, per-object RBAC (table- and column-level permissions).

References

[1] E. F. Codd, "A relational model of data for large shared data banks," Communications of the ACM, vol. 13, no. 6, pp. 377–387, Jun. 1970.

[2] I. Androutsopoulos, G. D. Ritchie, and P. Thanisch, "Natural language interfaces to databases — an introduction," Natural Language Engineering, vol. 1, no. 1, pp. 29–81, Mar. 1995.

[3] V. Zhong, C. Xiong, and R. Socher, "Seq2SQL: Generating structured queries from natural language using reinforcement learning," arXiv:1709.00103, 2017.

[4] X. Xu, C. Liu, and D. Song, "SQLNet: Generating structured queries from natural language without reinforcement learning," arXiv:1711.04436, 2017.

[5] T. Yu et al., "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task," in Proc. EMNLP, 2018, pp. 3911–3921.

[6] B. Wang, R. Shin, X. Liu, O. Polozov, and M. Richardson, "RAT-SQL: Relation-aware schema encoding and linking for text-to-SQL parsers," in Proc. ACL, 2020, pp. 7567–7578.

[7] X. V. Lin, R. Socher, and C. Xiong, "Bridging textual and tabular data for cross-domain text-to-SQL semantic parsing," in Findings of ACL: EMNLP, 2020, pp. 4870–4888.

[8] T. Scholak, N. Schucher, and D. Bahdanau, "PICARD: Parsing incrementally for constrained auto-regressive decoding from language models," in Proc. EMNLP, 2021, pp. 9895–9901.

[9] G. Katsogiannis-Meimarakis and G. Koutrika, "A survey on deep learning approaches for text-to-SQL," The VLDB Journal, vol. 32, no. 4, pp. 905–936, 2023.

[10] N. Rajkumar, R. Li, and D. Bahdanau, "Evaluating the text-to-SQL capabilities of large language models," arXiv:2204.00498, 2022.

[11] M. Pourreza and D. Rafiei, "DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction," in Advances in NeurIPS, vol. 36, 2023, pp. 36339–36348.

[12] D. Gao et al., "Text-to-SQL empowered by large language models: A benchmark evaluation," Proc. VLDB Endowment, vol. 17, no. 5, pp. 1132–1145, 2024.

[13] J. Li et al., "Can LLM already serve as a database interface? A big bench for large-scale database grounded text-to-SQLs (BIRD)," in Advances in NeurIPS Datasets and Benchmarks Track, vol. 36, 2023.

[14] Z. Hong et al., "Next-generation database interfaces: A survey of LLM-based text-to-SQL," arXiv:2406.08426, 2024.

[15] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey et al. (Llama Team, AI @ Meta), "The Llama 3 herd of models," arXiv:2407.21783, 2024.

[16] D. Abts et al., "Think fast: A tensor streaming processor (TSP) for accelerating deep learning workloads," in Proc. ACM/IEEE ISCA, 2020, pp. 145–158.

[17] W. G. J. Halfond, J. Viegas, and A. Orso, "A classification of SQL-injection attacks and countermeasures," in Proc. IEEE Int. Symp. Secure Software Engineering (ISSSE), 2006.

[18] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," Computer, vol. 29, no. 2, pp. 38–47, Feb. 1996.

[19] X. Wu, L. Xiao, Y. Sun, J. Zhang, T. Ma, and L. He, "A survey of human-in-the-loop for machine learning," Future Generation Computer Systems, vol. 135, pp. 364–381, 2022.

[20] V. Dibia and Ç. Demiralp, "Data2Vis: Automatic generation of data visualizations using sequence-to-sequence recurrent neural networks," IEEE Computer Graphics and Applications, vol. 39, no. 5, pp. 33–46, 2019.

[21] A. Narechania, A. Srinivasan, and J. Stasko, "NL4DV: A toolkit for generating analytic specifications for data visualization from natural language queries," IEEE Trans. Visualization and Computer Graphics, vol. 27, no. 2, pp. 369–379, Feb. 2021.

Appendix A: Publication Details

A technical paper based on this project, titled "Meridian Data: A Safe, Multi-Database Natural-Language-to-SQL Explorer with Human-in-the-Loop Control," has been formatted in IEEE conference style (minimum six pages) and is being prepared for submission to an IEEE International Conference on Computing / Data Engineering. The paper consolidates the architecture of Chapter 4, the implementation of Chapter 5, and the experimental results of Chapter 6, and includes the accompanying Turnitin plagiarism and AI-content reports.

Title: Meridian Data: A Safe, Multi-Database Natural-Language-to-SQL Explorer with Human-in-the-Loop Control.

Authors: Ayon Aryan, Hardik Goel, Aniket Kumar, Aditya Kumar; Guide: Prof. Ashwini Mathur.

Target venue: IEEE International Conference (Computing / Data Engineering track).

Status: Formatted; pending submission.

Appendix B: Certificate for External Internship

Not applicable. This mini project was carried out within the School of Computer Science and Engineering, RV University, Bengaluru, and was not undertaken as part of an external organizational internship. Consequently, no external completion certificate or ongoing-internship undertaking is enclosed.